Capítulo 2: Fundamentos de Git

Tutorial de Git basado en Pro Git (2.ª ed.) · Chacon y Straub · Elaborado por Ricardo · UNIAGRARIA

Introducción

Este capítulo cubre los comandos que se usan a diario: crear o clonar repositorios, registrar cambios, consultar el historial, deshacer errores, trabajar con remotos, etiquetar versiones y crear alias.

2.1 Obteniendo un repositorio Git

Hay dos formas de obtener un repositorio: inicializar uno en una carpeta existente o clonar uno que ya existe.

```
# Inicializar en una carpeta existente
cd mi-proyecto
git init

# Clonar un repositorio remoto
git clone https://github.com/usuario/repositorio.git
git clone https://github.com/usuario/repositorio.git mi-carpeta
```

git init crea el subdirectorio oculto .git con el esqueleto del repositorio. git clone descarga todo el historial y deja activada la rama principal con el remoto llamado origin.

2.2 Guardando cambios en el Repositorio

Cada archivo del directorio de trabajo puede estar sin seguimiento (untracked) o con seguimiento (tracked), y los archivos con seguimiento pueden estar sin modificar, modificados o preparados.

```
git status        # estado de los archivos
git status -s     # versión corta
git add notas.txt # empieza a seguir o prepara el archivo
git add .         # prepara todo lo modificado
git commit -m "Agrega notas.txt con apuntes de la clase 1"
```

Ignorar archivos

El archivo .gitignore lista patrones de archivos que Git no debe seguir (temporales, compilados, credenciales).

```
# .gitignore
*.log
*.tmp
/build/
!importante.log
```

Ver los cambios

```
git diff            # cambios sin preparar
git diff --staged    # cambios preparados (equivale a --cached)
```

Confirmar, eliminar y mover

```
git commit            # abre el editor para el mensaje
git commit -a -m "Corrige ortografía en notas.txt"   # salta el add de archivos ya
seguidos
```

```
git rm ideas.txt            # elimina del directorio y del índice
git rm --cached secretos.txt # deja de seguirlo pero lo conserva en disco
git mv tareas.txt pendientes.txt   # renombra
```

2.3 Ver el Historial de Confirmaciones

```
git log
git log --oneline
git log --oneline --graph --decorate --all
git log -p -2            # cambios (patch) de los últimos 2 commits
git log --stat           # resumen de archivos modificados
git log --pretty=format:"%h %an %ar %s"
git log --since=2.weeks
git log --author="Ricardo"
git log -- notas.txt     # historial de un archivo
```

Entre las opciones de formato más usadas: %h (hash corto), %an (autor), %ar (fecha relativa) y %s (asunto).

2.4 Deshacer Cosas

```
# Corregir el último commit (mensaje o archivos olvidados)
git commit --amend

# Sacar un archivo del área de preparación
git restore --staged notas.txt
# (forma antigua: git reset HEAD notas.txt)

# Descartar cambios locales de un archivo
git restore notas.txt
# (forma antigua: git checkout -- notas.txt)
```

> **Nota:** git restore y git checkout -- <archivo> descartan cambios sin posibilidad de recuperarlos; todo lo que ya fue confirmado, en cambio, casi siempre puede recuperarse.

2.5 Trabajar con Remotos

```
git remote -v                     # lista los remotos
git remote add origin https://github.com/usuario/repo.git
git fetch origin                   # descarga sin fusionar
git pull                           # fetch + merge de la rama actual
git push origin main               # envía los commits
git remote show origin             # detalle del remoto
git remote rename origin github
git remote remove github
```

git fetch solo trae los datos a la base local; git pull además intenta fusionarlos en la rama en la que se está trabajando.

2.6 Etiquetado

```
git tag                        # lista etiquetas
git tag -a v1.0 -m "Versión 1.0"   # anotada
git tag v1.0-ligera              # ligera
git tag -a v0.9 9fceb02          # sobre un commit anterior
```

```
git show v1.0
git push origin v1.0              # las etiquetas no se envían solas
git push origin --tags
git tag -d v1.0-ligera
```

Las etiquetas anotadas guardan autor, fecha y mensaje, y son las recomendadas para versiones; las ligeras son solo un puntero a un commit.

2.7 Alias de Git

```
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.st status
git config --global alias.last 'log -1 HEAD'
git config --global alias.lg "log --oneline --graph --decorate --all"
```

Resumen

El ciclo de trabajo básico es modificar, preparar (git add) y confirmar (git commit), consultando el estado con git status y el historial con git log. Es posible corregir errores con --amend y git restore, sincronizar con remotos mediante fetch, pull y push, marcar versiones con etiquetas y acortar comandos con alias.

Referencia

Chacon, S., & Straub, B. (2014). Pro Git (2nd ed.). Apress. https://git-scm.com/book/es/v2 — Capítulo 2.